

Autour du test

Attention : Version d'Eclipse à utiliser : Eclipse JEE. Cette version d'eclipse contient déjà gradle installé, donc inutile d'installer le greffon gradle.

Exercice 1 : Récupérer le projet fourni

Pour ce TP, nous allons utiliser gradle, un outil qui permet d'automatiser les tâches d'un projet comme la compilation, l'exécution, l'exécution des tests, le nettoyage, etc. gradle n'est pas installé sur les machines de l'N7 et il ne sera pas nécessaire de l'installer explicitement : le projet que nous vous fournissons contient une amorce gradle qui permettra de l'installer ainsi que tous les outils nécessaires.

1.1. Récupérer le fichier TPtest.tgz via Moodle.

1.2. Faire : `tar xzvf TPtest.tgz`

1.3. Aller dans le dossier TPtest et exécuter `./gradlew run`

La première exécution de cette commande sera longue car elle télécharge gradle et tous les éléments nécessaires pour compiler l'application. Une deuxième exécution de cette commande sera plus rapide

1.4. La commande `./gradlew clean` permet de nettoyer le projet.

Exercice 2 : Intégration de Gradle dans Eclipse

Voyons comment importer un projet Gradle existant sous Eclipse.

2.1. *Installer le greffon pour Gradle.*

Cette question n'est pas à faire car gradle est déjà installé sur l'eclipse JEE des stations ubuntu de l'ENSEEIH.

Sous Eclipse, faire *Help > Install New Software...*¹. Dans la boîte de dialogue, renseigner le champ « *Work with :* » avec le dépôt d'Eclipse (il est de la forme

oxygen - <http://download.eclipse.org/releases/oxygen/>

Oxygen qui dépend de la version d'Eclipse utilisée). La liste des greffons disponibles apparaît.

Dans la zone de saisie juste en dessous (*type filter text*), taper *gradle* et le greffon *Buildship : Eclipse Plug-ins for Gradle* apparaît seul. Le cocher et faire *Next*, Eclipse calcule les dépendances et affiche la liste des greffons à installer. Faire *Next* pour accéder à l'écran suivant. Lire la licence et l'accepter (ou la refuser, l'installation sera alors annulée).

2.2. *Importer un projet existant.* Il suffit de faire *Import...*, sélectionner *Existing Gradle Project* et suivre les instructions. En particulier, le *Project root directory* doit être le dossier racine du projet Gradle à importer.

1. on aurait pu aussi passer par le *Eclipse Marketplace* toujours dans le menu *Help*.

2.3. Ouvrir la vue des tâches Gradle. Faire *Window > Show View > Others...* et sélectionner *Gradle Tasks*. Cette vue présente sous forme arborescentes les tâches disponibles sous Gradle. Elle peuvent être lancées avec un double-click ou un click droit.

Exercice 3 : Utilisation d'un outil de couverture de code par les tests

Un outil de couverture de code par les tests peut être utilisé pour identifier des « morceaux » de code qui n'ont pas été testés. Nous allons utiliser la bibliothèque JaCoCo.

Nous prenons comme exemple la classe `Statistiques` qui contient une méthode `statistiques`. Cette dernière prend en paramètre un « itérable » pour lequel elle calcule la plus petite valeur, la plus grande, le nombre d'occurrences de la plus petite valeur et le nombre d'occurrences de la plus grande valeur. Son code contient des erreurs. Lire la documentation de cette méthode pour avoir plus de détails. On peut engendrer la documentation en lançant la tâche gradle `javadoc`. La documentation est engendrée dans `build/docs/javadoc`.

La méthode `statistiques` a été testée avec `StatistiquesTest`, classe de test bien incomplète !

3.1. Quel est l'objectif de la démarche de test (et des programmes de test) ?

3.2. Définir, sur papier, des cas de tests pour la méthode `statistiques`.

3.3. Le fichier de configuration de gradle fait apparaître `jacoco` dans les greffons (plugins). Quand les tests seront lancées (par exemple `./gradlew test` dans un terminal), l'analyse de couverture sera lancée.

Pour produire un rapport du calcul de couverture, on peut faire `./gradlew jacocoTestReport` qui engendre les résultats dans le dossier `build/reports/jacoco/test/html`. Le fichier `index.html` est le fichier à ouvrir dans un navigateur.

Lancer les tests et consulter le rapport de couverture. Ne considérer que `Statistiques.java`).

3.4. Compléter les tests pour atteindre un taux de couverture de 100% pour les instructions et les décisions. Attention, il ne faut pas modifier les tests existants mais ajouter de nouveaux tests !

3.5. Que peut-on conclure quand on a atteint un taux de couverture de 100% en instructions et décisions avec des tests qui réussissent ?

3.6. Est-ce que tous les cas de tests identifiés en 3.2 sont couverts par les tests effectués ? Dans la négative, ajouter des tests pour les couvrir.

Exercice 4 : Utilisation de doublures (mock) pour les tests

Certaines méthodes que l'on veut tester peuvent nécessiter d'autres objets qu'il n'est pas facile d'utiliser de manière réelle comme par exemple la connexion à un serveur, une base de données, etc. Dans ces cas, il peut être intéressant d'utiliser des doublures des objets réels (des *mock* en anglais) pour simuler le comportement des objets réels ou vérifier que les objets réels seront correctement utilisés.

Nous allons l'illustrer avec la méthode `somme` de la classe `Somme`. Cette méthode calcule la somme des entiers qui sont contenus dans le `BufferedReader` qui est fourni en paramètre. En cas d'erreur dans le fichier (une ligne qui ne correspond pas à un entier, ou un entier négatif), une erreur est signalée en utilisant l'interface `Erreurs`.

L'objectif est de tester le bon comportement de cette méthode aussi bien en ce qui concerne le traitement des informations du `BufferedReader` que le signalement des erreurs.

Au lieu de vrais fichiers, nous allons utiliser des doublures en utilisant `Mockito`.

Pour la suite, on utilisera la documentation sur Mockito qui est disponible à l'URL suivante : <http://static.javadoc.io/org.mockito/mockito-core/2.23.4/org/mockito/Mockito.html#1>

4.1. Lire, comprendre le test `testerSommeAvecUnVraiFichier`.

4.2. Lire, comprendre le test `testerSommeAvecUneDoublure`.

4.3. Exécuter les tests (`./gradlew test`) et, en cas d'erreur, consulter le rapport sur les tests qui est disponible via `build/reports/tests/test/index.html`

4.4. Ajouter un nouveau test en utilisant une doublure de `BufferedReader` qui produit les informations suivantes : "1", "-5", "2", "xxx", "4" et bien sûr `null` qui indique la fin du flot.

Le résultat attendu devrait être 7 car -5 sera ignoré car il est négatif et "xxx" aussi car il est invalide (pas un entier).

4.5. Ajouter un nouveau test avec une doublure qui lève une exception de type `IOException` après avoir fourni les valeurs "1", "-1", "2", "trois" et avant de fournir les valeurs "10", "20" et `null`.

Pour réussir, le test doit vérifier que la méthode `somme` laisse se propager l'exception `IOException`.

Serait-il facile de réaliser ce test avec un vrai fichier ?

4.6. Compléter l'avant dernier test (somme vaut 7) pour vérifier que les erreurs sont correctement signalées. On utilisera une doublure de l'interface `Erreurs` !

4.7. Vérifier que les erreurs sont signalées dans le bon ordre.

Exercice 5 : Évaluer la qualité d'une suite de test (avec Pitest)

Pour évaluer la qualité des programmes de test, une stratégie consiste à modifier le programme sous test (par exemple changer un « < » en « > », un « + » en « - », etc.) et à vérifier que les tests détectent l'erreur. Le programme modifié est obtenu par « mutation » du programme sous test et est appelé « mutant ». Si les tests signalent une erreur (et donc échouent) sur le mutant, c'est que ce sont des bons tests. Si les tests réunissent, on dit que le mutant « survit » et on en déduit que les tests sont incomplets.

Pitest est un outil qui réalise ce type de test.

5.1. *Exécuter pitest.* La configuration pour pitest a déjà été faite. Pour exécuter pitest, il suffit donc de faire :

```
./gradlew pitest
```

5.2. *Consulter les résultats.* Les résultats sont produits dans le dossier `build/reports/pitest/` avec un sous-dossier par lancement de pitest qui correspond à la date du lancement. On peut alors ouvrir le fichier `index.html` dans un navigateur.

5.3. *Compléter les tests.* Compléter les tests pour essayer de tuer les mutants survivants.